EVM compatibility

Copy page

Arc EVM compatibility details covering USDC as gas, deterministic sub-second finality, and key differences for developers.

Arc is compatible with the Ethereum Virtual Machine (EVM). Developers can deploy and interact with smart contracts using the same tools, languages, and frameworks they use on Ethereum such as Solidity, Foundry, and Hardhat.

While the execution environment mirrors Ethereum's, Arc introduces a few key differences:

- **USDC as native gas:** All fees and balances are denominated in USDC, not ETH.
- **Deterministic finality:** Transactions finalize instantly and cannot be reversed.
- **Simplified block times:** Blocks are timestamped by real time, not epochs or slots.
- **Stable fee model:** Gas prices are smoothed for predictability.
- **Permissioned validators:** Arc uses a BFT consensus model (Malachite) for speed and reliability.

These changes make Arc predictable and stable while preserving EVM compatibility.

ERC20 interface

Arc uses USDC as its native network token. Native balances behave like ETH on Ethereum and are represented with 18 decimals. An optional ERC20 interface is also available: see **USDC contract address**. This ERC20 provides the same capability as USDC on other EVM networks, such as ERC20 allowances and `transferFrom`, and uses **6 decimals** to match the standard USDC representation.

This has two important effects:

1. Tiny USDC amounts (less than 1×10^{-6} USDC) cannot be transferred using the ERC20 interface.

2. Protocols that hold USDC as an ERC20 automatically hold equivalent native balances. No additional Solidity changes are required (for example, `payable` or `receive` functions), as ERC20 transfers are directly reflected in the native balance.

EVM differences

Arc targets the **Prague** EVM hard fork with minor differences in execution and consensus behavior.
The table below summarizes key distinctions from Ethereum mainnet.

Area	Ethereum	Arc
Native token	ETH, volatile pricing	USDC, stable pricing with 18 decimals; used as gas
Fee market	EIP-1559 base fee per block	Fee smoothing with moving average; stable, bounded base fee, inspired by EIP-1559
Finality	Probabilistic (≈12–15 min for safety)	Deterministic and instant (<1 s)
Consensus	Proof-of-Stake (slot/epoch model)	Malachite (Tendermint-based) BFT with permissioned validators
Block timestamps	Derived from slots and epochs	Wall-clock time from proposer with second-level granularity; sub-second blocks may share timestamps
`SELFDESTRUCT`	Allowed with value transfers to self	Not allowed during deployment to prevent burning native tokens
`PARENT_BEACON_BLOCK_ROOT`	Root of parent beacon block (SSZ)	Hash of parent execution payload header (`keccak256(RLP(header))`); no beacon chain
`PREV_RANDAO`	Randomness mix of proposer reveals	Always `0` ; not used for randomness
USDC blocklist handling	Runtime revert on transfer	Pre-block inclusion check when possible; reverts or blocks as described below
EIP-4844 blobs	Supported post-Dencun	Currently disabled

USDC blocklist revert handling

Arc enforces USDC blocklists both pre- and post-execution:

- **Pre-mempool check:** If the sender is blocklisted, the transaction is rejected before entering the mempool. No fees are collected.
- **Post-mempool check:** If the address becomes blocklisted after acceptance but before execution, the transaction reverts at runtime and consumes gas.
- **Runtime transfer check:** If a valid transaction attempts to move USDC to or from a blocklisted address, only that operation reverts. Fees are still collected.

Developer impact

For most use cases, Ethereum-based tooling and smart contracts will work on Arc without modification. However, developers should note a few practical differences:

- **Gas denomination:** All values are in USDC. When using libraries like `ethers.js` or `viem`, display and accounting logic should format values in USD terms, not ETH.
- **Timestamps:** Multiple blocks may share the same timestamp; avoid assuming strictly increasing values for onchain time comparisons.
- **Randomness:** `block.prevrandao` is always zero. Do not use it as a source of randomness; use an external oracle or verifiable randomness function (VRF) instead.
- **Finality:** Transactions finalize immediately after inclusion. Offchain systems can safely act on events after a single confirmation: no additional block waiting is required.
- **SELFDESTRUCT restrictions:** Contracts that self-destruct during deployment will revert if they attempt to send USDC value to themselves.

These differences are designed to make Arc's execution environment predictable and stable while remaining interoperable with the Ethereum developer ecosystem.

Was this page helpful? Yes No

‹ Deploy on Arc Running a node ›

Links

Network status

Blog

Community

Arc homepage

Resources

Terms of service

Privacy policy

Cookie policy

Powered by mintlify

By using the Arc Network Testnet, you agree to the Terms of Service and Privacy Policy.